

Git Case Study: In-Class Exercise

Objective

As a team, you are tasked with recreating the case study that we've seen on slides 17-37 in [Intro to Git](#). In teams of 3, you will assume the roles of Eric, Lucy, and Adam. Each team member will recreate the steps that their corresponding roles completed to exercise the following skills:

- Using the terminal to run git commands
- Creating branches
- Creating new files
- Add changed files to your commits
- Resolving merge conflicts
- Approving merge requests
- Completing merge requests

Keep in mind that the focus is *not* to get the JavaScript files working, but to focus on getting Git correctly set up and to emulate the same Git processes as the developers in the case study. Focus on Git and Github.

Note: If one team only has a team of 2, TA's may fill in for the third team member. Also, **instructors**, please keep in mind the "Git Configuration" section.

All instructions below are to be followed sequentially. This exercise may take up to 30-45 minutes to complete so please be mindful of the allotted time.

Creating Your Repo

1. All team members: If you have not already done so, create your GitHub profile and ensure that you are logged in.

- a. Ensure you have completed your Git Configuration. You can test this from your PC's terminal by running the '***git log -list***' command. It will notify you which authorized account is tied to your terminal.

- b. If you are receiving an error requesting the ``git config`` command, you may need to complete your Git Configuration. Notify your instructor immediately and see the Git Configuration section before moving onto **step 2.0**.

2. Only one team member: From your Home page, press the “+” at the top right of the page and click ‘**New Repository**’.

- a. Name your repo something relevant to your assignment
- b. Make it **Public**
- c. Click ‘**Create Repository**’ at the bottom of the page.
- d. Once your repo is created, go to ‘Settings and under ‘**Collaborators**’, add each of your teammates
 - They will need to accept the invitation via the email connected to their Github account
- e. In **Visual Studios**, create a new folder for your project and make sure your terminal is pointing at the correct folder before moving on.
- f. In Visual Studios, open your terminal and run the commands found under “...or create a new repository on the command line”. This will do the following:
 - Initialize your repository.
 - Add and commit a README.md file to your repo
 - Renames the “master” branch to “main”
 - Creates a new remote repository based on the provided URL
 - Pushes the changes made (see: ii)
- g. On your repository’s web page, click the **Branch** button, then click ‘**View All Branches**’
 - On the redirected page, click the ‘**New Branch**’ button
 - Name it ‘**Develop**’

Again, if you are receiving an error requesting the ``git config`` command, you may need to complete your Git Configuration. Notify your instructor immediately and see the Git Configuration section.

Otherwise, upon refreshing your page, you should see a new layout with your README added to your commit history, meaning your repository is ready for cloning. Continue to the Add Rules to Your Main Branch section.

Git Configuration

This section only applies if you have not completed your Git configuration in your terminal. This is apparent if running the commands under “...or create a new repository on the command line” results in an error requesting either a ``user.email`` or ``user.name``.

If this applies to you or one of your team members, please notify the instructor right away. It is required for every student to have their Git configuration completed before moving forward.

Instructors: Complete [Git Configuration](#) with fellows during class only if they have not done so previously.

- If students amongst a given group *have* completed their Git configuration, either have them work together to complete the case study, add them to a pre-existing team, or pair them with the TA (if available).

If you (fellow) were not able to complete the case study during class, it is recommended you attend office hours to do so with your other classmates or your instructional staff. Please notify the instructor before the end of class if that is the case so that your instructional team can make notes of it.

Add Rules to Your Main Branch

To prevent any major obstacles in the development cycle, we will add rules to our Main branch.

1. One team member: On the Settings page of your repository is a section called ‘Branches’

a. Add a protection rule to our ``main`` branch. Click ‘**Add rule**’

b. Under ‘**Branch name pattern**’, enter ``main``

- The ‘*branch name pattern*’ needs to be identical to the branch you are adding protections. Otherwise, you may receive an ‘invalid’ error

c. Add the ‘**Require a pull request before merging**’ and ‘**Require approvals**’ rules just

below. Set the required amount to **1**

- This ensures that any changes being made will first require approval from another collaborator before being merged into the `main` branch.
 - This also protects the **production** version of your application from receiving errors or buggy changes.
-

Cloning Your Repository

The team member who created the repository should see a **Code** button above their commit history. From here your team will be able to clone the repository your team will be working on.

Under the **Code** button, you'll see a **Clone** header, and below that, you'll see **'HTTPS'**.

1. Copy the URL provided
2. In Visual Studio Code, run the 'git clone' command with provided URL at the end
 - Example: `git clone https://github.com/randomuser/randomrepository.git`

If you receive any errors, take the time to research what may need to be done to resolve them. ***All team members should work together to resolve whatever errors are affecting your team's progress.***

When successfully done, all team members should have a local copy of the team's project. This is apparent when you can see the README.md *and* .git file in your project folder.

Note: DO NOT simply move the README.md and .git file if they are outside your project folder. This may create unnecessary and avoidable complications. If there is any confusion, try reaching out to the instructional staff if they are not presently working with other students.

Feature-A: Team Member "Eric"

Hi Eric, you'll be responsible for greeting users and checking their names.

Please share your screen with your team so they can observe your process. If you do not have Share Screen permissions, please notify the host of your Zoom call.

Since you have already cloned your repository, you can skip that first step.

1. You should check which branch you're presently on and run the following commands:

None

```
git branch  
git switch develop
```

Note: If you cloned BEFORE the 'develop' branch was created, you will need to run `git pull` first in order to retrieve the branch from your remote repository into your local repository. Otherwise, simply switch branches.

2. Create and switch to your Feature-A branch by running the following commands:

None

```
git checkout -b feature-a
```

3. Add two new files and add the code from [slide 26](#) to the following files:

JavaScript

```
// welcome.js  
  
import { verifyName } from './getName.js';  
  console.log("=".repeat(35));  
  
console.log("Welcome to eligibility check");  
  
console.log("=".repeat(35));  
  
console.log("\n");  
  const userName = verifyName(username);  
  
console.log(`Hello ${userName}!\n`);
```

JavaScript

```
// getName.js  
  
export function verifyName() {  
  const input = prompt("Enter your age: ");
```

```
    return input;  
}
```

4. Add your changed files to your local repo by running the following command:

None

```
git add welcome.js getName.js
```

5. Check the status of your files by running the following command:

None

```
git status
```

You should see your files are ready to commit in your terminal.

6. Create your first commit on the Feature-A branch with a description of the commit.

None

```
git commit -m "([Your Name]) Feature-A: Show a greeting  
message to user and ask for their name"
```

- Use your first name and last initial to indicate who is playing the role of Eric.
- Example: **“(Amy A) Feature-A: Show a greeting message to user and ask for their name”**
- **DO NOT copy and paste the message above.** You may get insane errors. Make sure to type it out manually.

7. In order to see your changes on your remote branch, run the following command:

None

```
git push --set-upstream origin feature-a
```

If done correctly, you should see a prompt that will ask you to 'Compare & pull request' once you've refreshed your webpage.

8. Check that your remote repository reflects your commits. ([See slide 32](#))

9. Compare and approve the pull request ([See slide 33](#))

- Make sure on the '**Comparing changes**' page, you switch the base branch from '**main**' to '**develop**'. This will merge the changes from your feature branch onto the develop branch. Remember, this is how we can test, and quality assure our features before they are pushed into production and become client-facing

If at any point you are encountering unique errors, work together as a team to resolve them. Only reach out to instructional staff if they are not already assisting other students *and* you've done extensive research to find a solution.

Feature-B: Team Member “Lucy”

Hi Lucy, you'll be responsible for Feature-B, verifying users' ages.

Please share your screen with your team so they can observe your process. If you do not have Share Screen permissions, please notify the host of your Zoom call.

Since you have already cloned your repository, you can skip that first step.

1. You should check which branch you're presently on by running the following command:

None

```
git branch
```

2. Create and switch to your Feature-B branch by running the following command:

None

```
git checkout -b feature-b
```

3. Add two new files and add the following code:

JavaScript

```
// welcome.js

import { validateAge } from "./validateAge.js";

const age = validateAge(age);
```

JavaScript

```
// validateAge.js

export function validateAge(age) {

    age = parseInt(prompt("Enter your age: "), age);

    if(age < 18){

        console.log('Not eligible for voting\n\n');

    } else {

        console.log('Eligible for voting\n\n');

    }

}
```

3. Create your first commit on the Feature-B branch with a description of the commit.

None

```
git add welcome.js validateAge.js

git commit -m "([Your Name]) Feature-B: Ask the user for their age. A user is only eligible to vote if their age is greater than or equal to 18"
```

- Use your first name and last initial to indicate who is playing the role of Eric.
- Example: **"(Bob B) Feature-B: Ask the user for their age. A user is only eligible to vote if their age is greater than or equal to 18"**.

- ***DO NOT*** copy and paste the message above. You will get insane errors. Make sure to type it out manually.

4. In order to see your changes on your remote branch, run the following command:

None

```
git push --set-upstream origin feature-b
```

- If done correctly, you should see a prompt that will ask you to 'Compare & pull request' once you've refreshed your page.

5. Check your **Pull Requests** tab on your repository's web page and check for merge conflicts

- Make sure on the '**Comparing changes**' page, you switch the base branch from '**main**' to '**develop**'. This will merge the changes from your feature branch onto the develop branch. Remember, this is how we can test, and quality assure our features before they are pushed into production and become client-facing.
 - After creating the pull request, you should get an error prompt about a merge conflict with '**welcome.js**'. Click '**resolve conflict**'.

6. Fix the **welcome.js** file using the code from [slide 41](#)

- Recall slides [42-43](#) to observe **how** the changes have been made.

JavaScript

```
// welcome.js

import { validateAge } from "./validateAge.js";
import { verifyName } from './getName.js';

console.log("=".repeat(35));
console.log("Welcome to eligibility check");
console.log("=".repeat(35));
console.log("\n");
```

```
const age = validateAge(age);  
const userName = verifyName(username);  
console.log(`Hello ${userName}!\n`);
```

7. Recommit the **welcome.js** file by clicking **Mark as resolved** and **Commit merge**.

8. Complete the pull request.

If at any point you are encountering unique errors, work together as a team to resolve them. Only reach out to instructional staff if they are not already assisting other students *and* you've done extensive research to find a solution.

Feature-C: Team Member “Adam”

Hi Adam, your objective is to create your own branch, merge the changes onto the Develop branch, then merge the Develop branch onto the Main branch. Be mindful of the changes that both Eric and Lucy have made already.

Please share your screen with your team so they can observe your process. If you do not have Share Screen permissions, please notify the host of your Zoom call.

Since you have already cloned your repository, you can skip that first step.

1. Pull the changes made by Eric and Lucy into your Feature-C branch by running the following command:

None

```
git pull origin develop
```

2. Create your new files and copy the following code.

JavaScript

```
// deliverSurvey.js  
  
export function deliverSurvey() {
```

```
    const input = prompt("Would you like to participate in  
    our survey (Y/N): ");  
  
    return input;  
}
```

3. Edit the following file (Do not add duplicate code. Only add what is new):

JavaScript

//welcome.js

```
import { validateAge } from "./validateAge.js";  
import { verifyName } from './getName.js';  
import { deliverSurvey } from "./survey.js";  
  
console.log("=".repeat(35));  
console.log("Welcome to eligibility check");  
console.log("=".repeat(35));  
console.log("\n");  
  
const age = validateAge(age);  
const userName = verifyName(username);  
console.log(`Hello ${userName}!\n`);  
  
const survey_flag = prompt("Would you like to participate  
in our survey (Y/N): ")  
if(survey_flag == 'Y')
```

```
deliverSurvey()
```

4. Add your changed files to your local repository using the following command:

None

```
git add deliverSurvey.js welcome.js
```

5. Create your first commit on the Feature-C branch with a description of the commit.

None

```
git commit -m "([Your Name]) Feature-C: At the end, ask the user to participate in a simple survey to determine their experience (on a scale of 1 to 5)"
```

- Use your first name and last initial to indicate who is playing the role of Eric.
- Example: **"(Carl C) Feature-C: At the end, ask the user to participate in a simple survey to determine their experience (on a scale of 1 to 5)"**.

6. In order to see your changes on your remote branch, run the following command:

None

```
git push --set-upstream origin Feature-C
```

If done correctly, you should see a prompt that will ask you to **'Compare & Pull Request'** once you've refreshed your page.

7. Check your **Pull Requests** tab on your repository's web page and check for merge conflicts.

- Make sure on the '**Comparing changes**' page, you switch the base branch from 'main' to 'develop'.
- Remember, this is how we can test, and quality assure our features before they are pushed into production and become client-facing.

8. Complete the pull request.

9. Switch to the **Main** branch on your remote repository after refreshing the page. Select '**Compare & Pull Request**'.

10. Merge the **Develop** branch into the **Main** branch with *no errors or merge conflicts*.

11. Ensure all of your team members' changes are reflected in the **Main** branch's commit history.

If successful, congrats team! You've successfully created your first team project using Git!